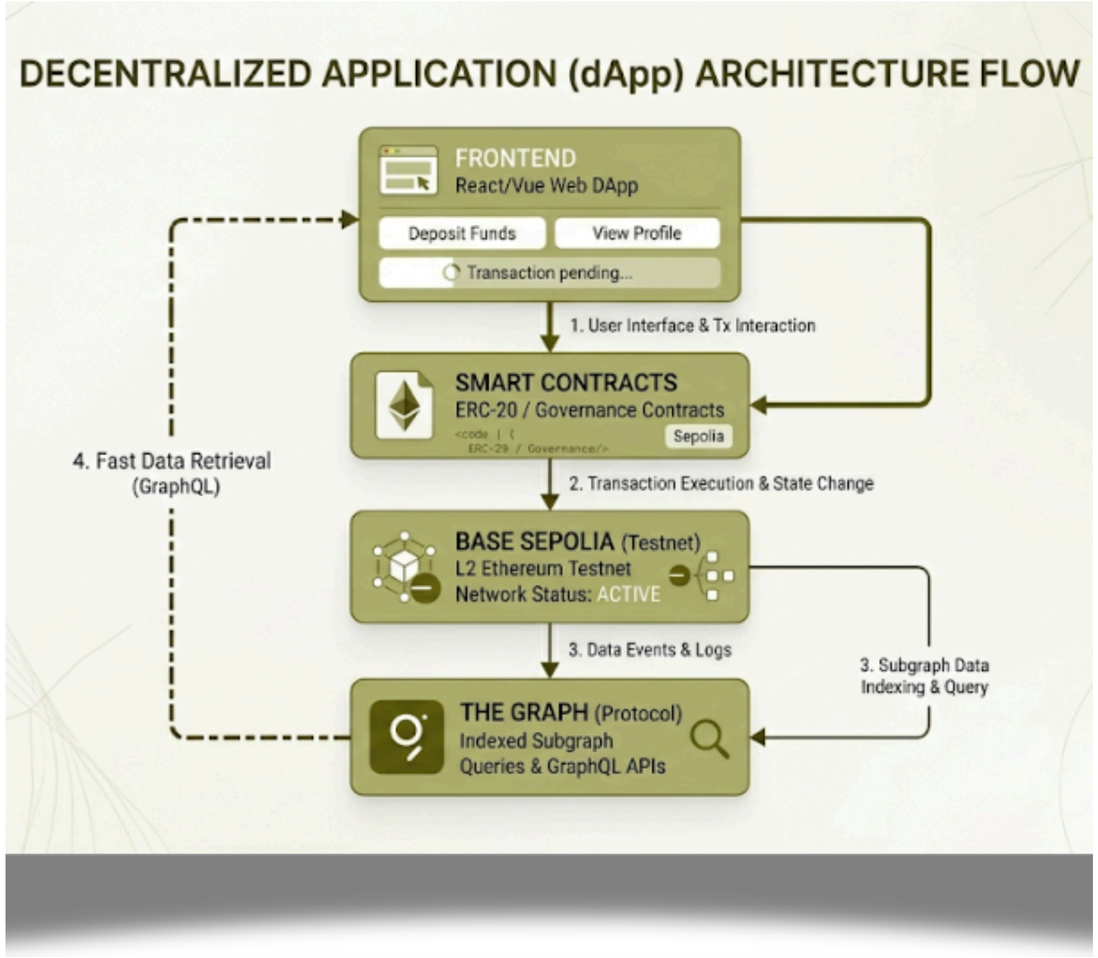
ethereum

BUILT WITH SOLIDITY, REACT, FOUNDRY,
THE GRAPH, AND BASE SEPOLIA.

RWA DAO TOKENIZATION PLATFORM

2026

ethereum



PROJECT OVERVIEW

A FULL-STACK DAO GOVERNANCE AND RWA TOKENIZATION PROTOCOL BUILT ON BASE SEPOLIA

Our project is a decentralized blockchain platform focused on governance, asset management, and Layer 2 scalability.

The system combines smart contracts, DAO voting, vault deposits, The Graph indexing, and a React frontend into one integrated DeFi application.

ethereum

KEY FEATURES

01 DAO GOVERNANCE SYSTEM
ERC20VOTES + ERC20PERMIT

02 BASE SEPOLIA DEPLOYMENT
REACT + WAGMI FRONTEND

03 ERC4626 VAULT
THE GRAPH INDEXING

04 SECURITY TESTING
UPGRADEABLE CONTRACTS

Built using Solidity, Foundry, OpenZeppelin, React, Viem, Wagmi, and The Graph.

ethereum

PROBLEM & SOLUTION

01

PROBLEMS

Centralized decision-making in traditional systems
Expensive gas fees on Ethereum Mainnet

02

PROBLEM

Difficult access to transparent governance
Slow and inefficient blockchain data retrieval

01

SOLUTION

DAO-based decentralized governance
Layer 2 deployment on Base Sepolia
ERC20Votes voting delegation

02

SOLUTION

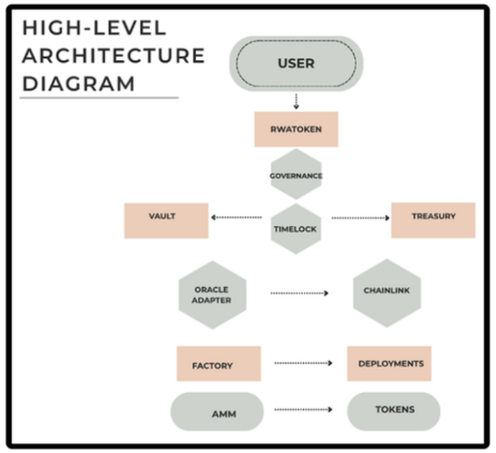
The Graph indexing for fast data queries
ERC4626 standardized vault system

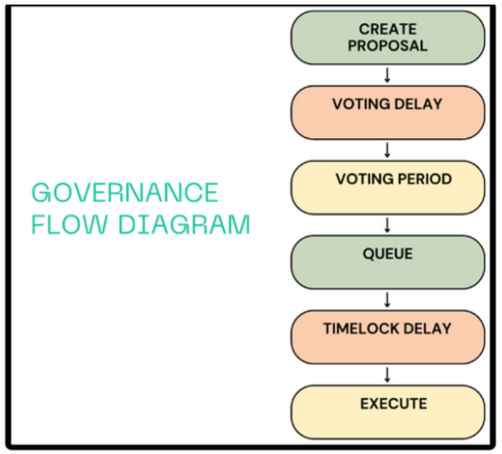
The platform was designed to simulate a scalable and transparent DeFi governance ecosystem.

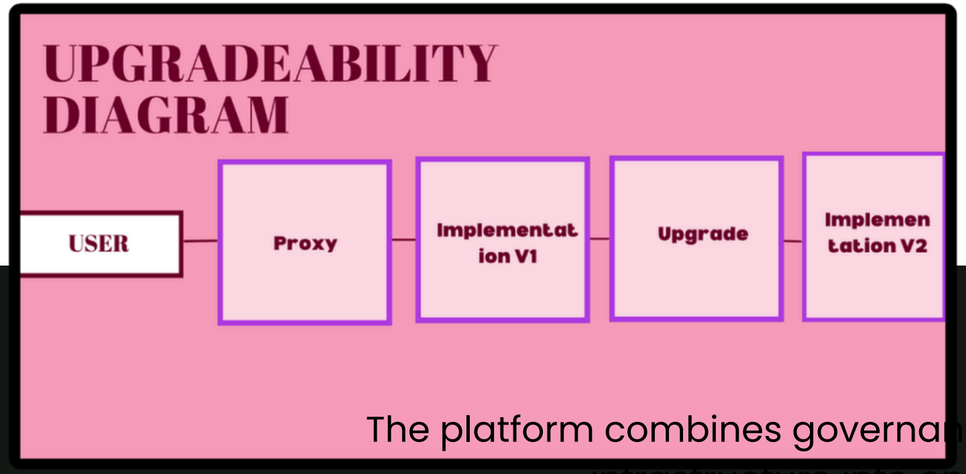
ethereum

SYSTEM ARCHITECTURE

THE PLATFORM COMBINES GOVERNANCE, VAULT MANAGEMENT, UPGRADEABLE SMART CONTRACTS, CHAINLINK INTEGRATION, AND LAYER 2 INFRASTRUCTURE INTO ONE DECENTRALIZED ECOSYSTEM.







Main Components

Governance System

ERC4626 Vault

Chainlink Oracle

Upgradeable ontracts

AMM + Factory Contrcts

Base Sepolia Deploymnt

The platform combines governance, vault management, upgradeability, and Layer 2 infrastructure into one integrated decentralized ecosystem.

ethereum

```
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Votes.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

contract RWAToken is ERC20, ERC20Permit, ERC20Votes, AccessControl {
    bytes32 public constant ISSUER_ROLE = keccak256("ISSUER_ROLE");

    constructor()
        ERC20("RWA Token", "RWA")
        ERC20Permit("RWA Token")
    {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(ISSUER_ROLE, msg.sender);

        _mint(msg.sender, 1_000_000 ether);
    }

    function mint(address to, uint256 amount)
        external
        onlyRole(ISSUER_ROLE)
```

```
function _update(
    address from,
    address to,
    uint256 value
) internal override(ERC20, ERC20Votes) {
    super._update(from, to, value);
}

function nonces(address owner)
    public
```

RWAToken Contract

FEATURES

ERC20Votes Governance Token

ERC20Permit Support

Delegated Voting Power

OpenZeppelin Implementation

Checkpoint-Based Governance

RWATOKEN POWERS THE DAO GOVERNANCE SYSTEM BY ENABLING TOKEN-BASED VOTING, DELEGATION, AND PROPOSAL PARTICIPATION.

ethereum

DAO Governance

FEATURES

OpenZeppelin Governor

TimelockController

Proposal State Tracking

Delegated Voting System

GOVERNANCE LIFECYCLE

1. Create Proposal
2. Voting Delay
3. Community Voting
4. Queue Proposal
5. Timelock Execution

The platform was designed to simulate a scalable and transparent DeFi governance ecosystem.

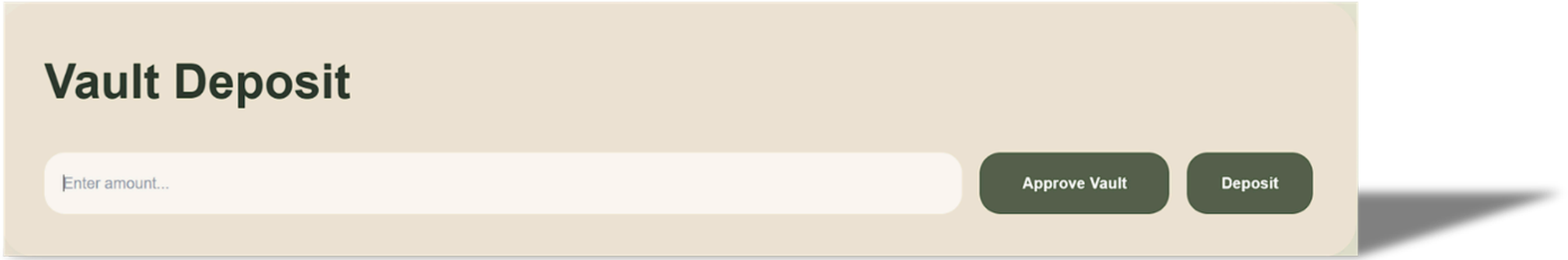
ethereum

Features

ERC4626 Tokenized Vault
Asset Deposits and Withdrawals
Standardized Vault Accounting
Approve → Deposit Transaction Flow

ERC4626 VAULT

The vault allows users to deposit governance tokens into a standardized ERC4626 asset management system.



ethereum

FRONTEND APPLICATION

The frontend provides a complete interface for interacting with governance and vault functionality directly from the browser.

Technologies

React

Wagmi

Viem

MetaMask Integration

Features

Wallet Connection

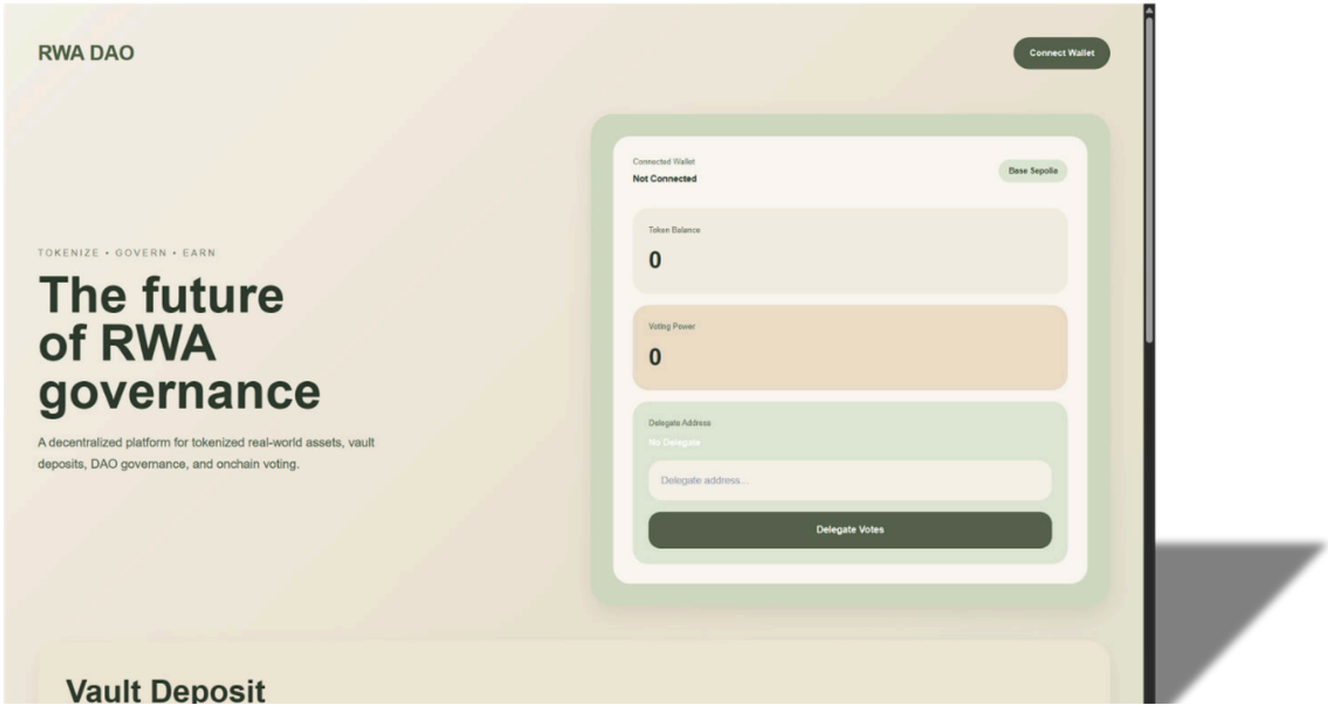
Governance Interface

Proposal Creation

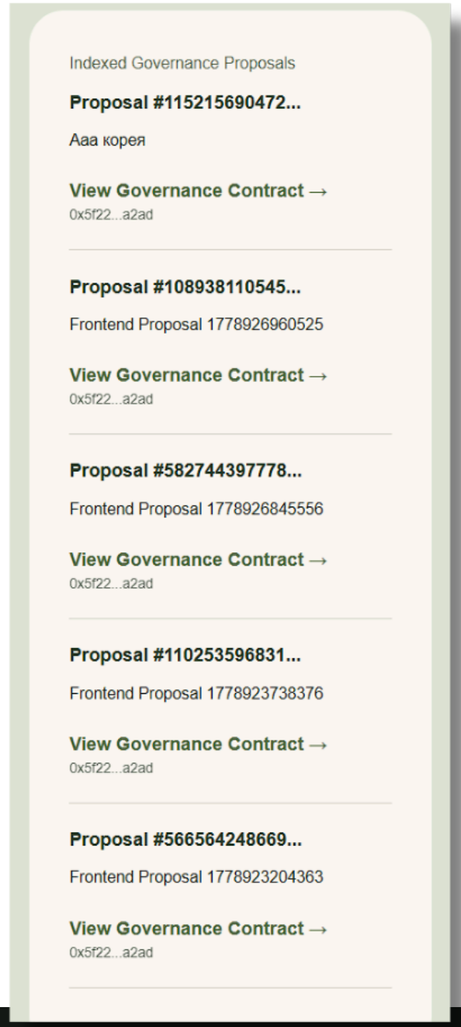
Voting System

Vault Deposits

Network Detection



ethereum



THE GRAPH INTEGRATION

The Graph indexes blockchain events and allows the frontend to retrieve governance activity efficiently through GraphQL queries.

Indexed Entities

- PROPOSAL
- VOTE
- DEPOSIT
- DELEGATECHANGE

Features

- REAL-TIME EVENT INDEXING
- FAST GRAPHQL QUERIES
- INDEXED GOVERNANCE DATA
- FRONTEND DATA SYNCHRONIZATION

ethereum

SECURITY & AUDIT

01

SECURITY MEASURES

OpenZeppelin AccessControl
TimelockController Protection
Reentrancy Protection
Checks-Effects-Interactions Pattern

02

SECURITY TESTING

Slither Static Analysis
Unit Testing
Fuzz Testing
Invariant Testing
Fork Testing

02

AUDIT RESULT

0 Critical Findings
0 High Findings
0 Medium Findings

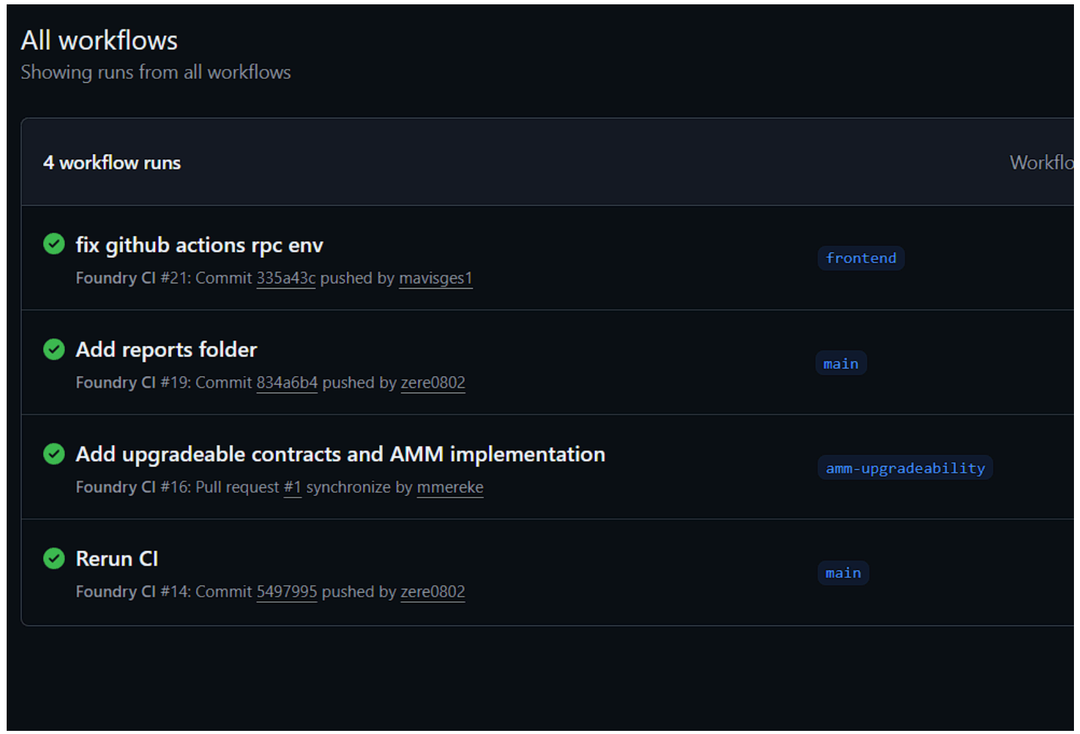
```
dikok@Dilnaz MINGW64 ~/OneDrive/Desktop/rwa-tokenization-platform (frontend)
$ slither .
'forge clean' running (wd: C:\Users\dikok\OneDrive\Desktop\rwa-tokenization-platform)
'forge config --json' running
'forge build --build-info --skip ./test/** ./script/** --force' running (wd: C:\Users\dikok\OneDrive\Desktop\rwa-tokenization-platform)
INFO:Detectors:
```

```
INFO:Detectors:
Detector: incorrect-exp
Math.mulDiv(uint256,uint256,uint256) (lib/openzeppelin-contracts/contracts/utils/math/Math.sol#201-271) has bitwise-xor operator ^ instead of t
he exponentiation operator **:
        - inverse = (3 * denominator) ^ 2 (lib/openzeppelin-contracts/contracts/utils/math/Math.sol#256)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-exponentiation
```

```
INFO:Detectors:
Detector: incorrect-equality
TimelockController.getOperationState(bytes32) (lib/openzeppelin-contracts/contracts/governance/TimelockController.sol#196-209) uses a dangerous
 strict equality:
        - timestamp == 0 (lib/openzeppelin-contracts/contracts/governance/TimelockController.sol#202-203)
TimelockController.getOperationState(bytes32) (lib/openzeppelin-contracts/contracts/governance/TimelockController.sol#196-209) uses a dangerous
 strict equality:
        - timestamp == DONE_TIMESTAMP (lib/openzeppelin-contracts/contracts/governance/TimelockController.sol#205)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dangerous-strict-equalities
```

```
INFO:Detectors:
Detector: shadowing-local
Governance.constructor(IVotes,TimelockController)._token (contracts/governance/Governance.sol#21-22) shadows:
        - GovernorVotes._token (lib/openzeppelin-contracts/contracts/governance/extensions/GovernorVotes.sol#15-16) (state variable)
Governance.constructor(IVotes,TimelockController)._timelock (contracts/governance/Governance.sol#22-23) shadows:
        - GovernorTimelockControl._timelock (lib/openzeppelin-contracts/contracts/governance/extensions/GovernorTimelockControl.sol#24-25) (sta
te variable)
ERC20Permit.constructor(string).name (lib/openzeppelin-contracts/contracts/token/ERC20/extensions/ERC20Permit.sol#37-38) shadows:
        - ERC20.name() (lib/openzeppelin-contracts/contracts/token/ERC20/ERC20.sol#50-52) (function)
        - IERC20Metadata.name() (lib/openzeppelin-contracts/contracts/token/ERC20/extensions/IERC20Metadata.sol#13-15) (function)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#local-variable-shadowing
```

ethereum



TESTING & CI/CD

Testing Types

- UNIT TESTS
- FUZZ TESTS
- INVARIANT TESTS
- FORK TESTS

CI/CD Pipeline

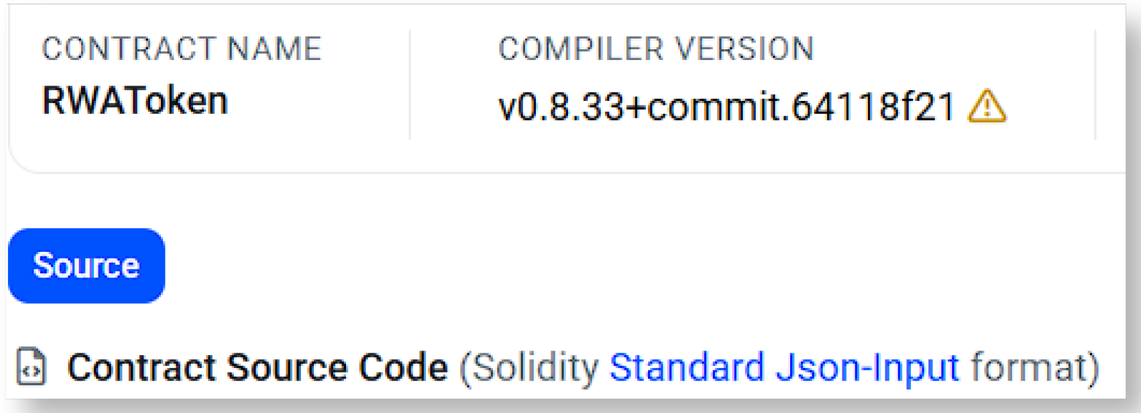
- FORGE BUILD
- FORGE TEST
- FORGE COVERAGE
- SLITHER ANALYSIS

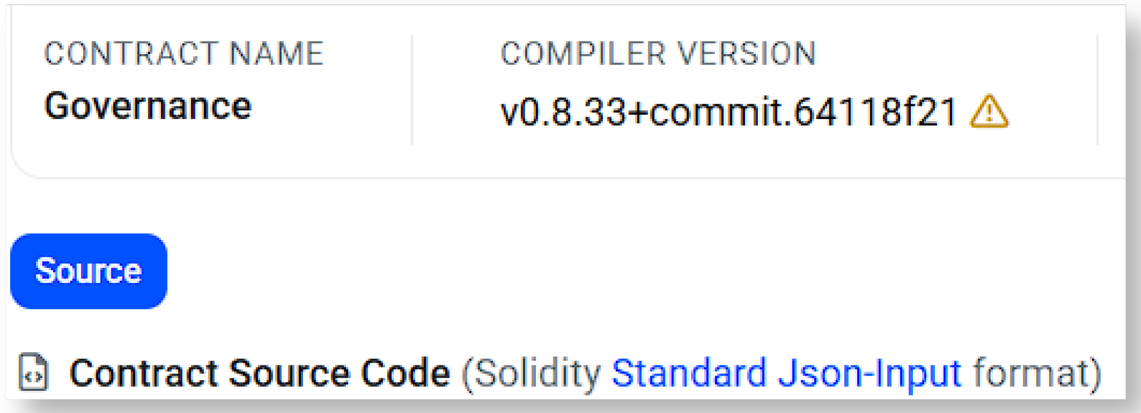
Coverage

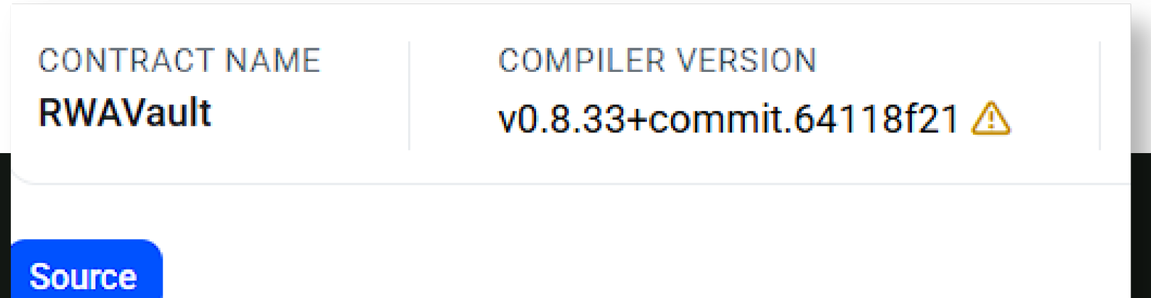
- 94% LINE COVERAGE
- 93% STATEMENT COVERAGE
- 92% FUNCTION COVERAGE

GitHub Actions automatically validates builds, tests, coverage, and security analysis on every push.

ethereum







Layer 2 Deployment

Deployment Network: Base Sepolia Layer 2

Benefits

- LOWER GAS FEES
- TRANSACTIONS
- SCALABLE GOVERNANCE
- IMPROVED USER EXPERIENCE

ethereum

Gas Optimization

OPTIMIZATION TECHNIQUES

ERC20Votes Checkpoints

Reduced Storage Writes

ERC4626 Standardization

Layer 2 Deployment

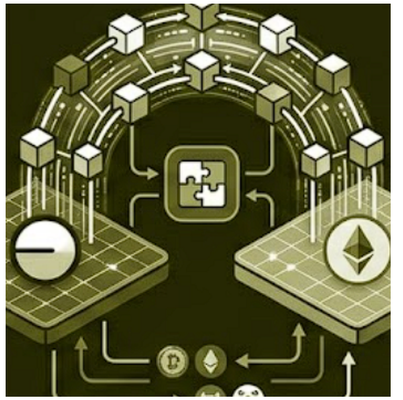
Yul Benchmarking

Gas Comparison

Operation	Ethereum	Base
Vote	Expensive	Cheap
Deposit	Medium	Cheap
Transfer	Medium	Cheap

ethereum

Challenges We Solved



Learnings

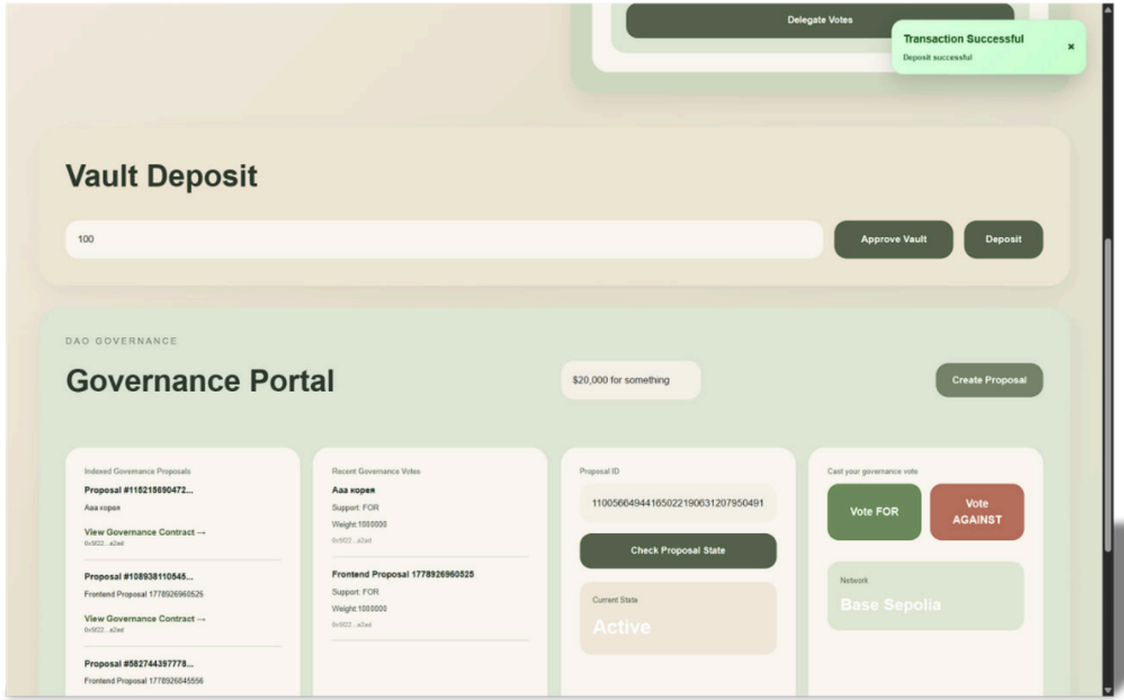
The project helped us gain practical experience in DAO governance, Solidity security, Layer 2 deployment, and full-stack blockchain development.

Challenges

We faced challenges while implementing governance delegation, vault transaction flows, and The Graph indexing integration.

ethereum

FINAL RESULT



Achievements

DAO Governance System
ERC4626 Vault
The Graph Integration
Security Testing & Audit
Layer 2 Deployment
React Frontend Application

The final result is a fully functional decentralized protocol combining governance, vault management, indexing, and Layer 2 infrastructure.

ethereum

THANK YOU

2026